



UNIVERSIDADE
FEDERAL DO CEARÁ
CAMPUS QUIXADÁ

Gerência de Configuração

[Git] Merge

Profa. Lana Mesquita



Conteúdo

- git merge
- Conflitos de merge
- git rebase
 - git pull --rebase
- Revisão merge e rebase
- git stash
- git cherry-pick

git merge

git merge

git-merge - Une dois ou mais históricos de desenvolvimento. Incorpora as alterações mencionadas dos commits (desde o momento onde seus históricos divergiram do ramo atual) no ramo atual.

O comando **git pull** incorpora um *git merge*.

git merge outra-branch

git merge --squash

A Standard Merge



A Merge using "--squash"



Conflitos de merge

git merge

Um merge pode gerar ou não conflito. Se conseguir resolver automaticamente não precisa resolver o conflito.

- Ao iniciar um conflito você não estará mais na branch atual. Entrará num espaço dedicado para resolver o conflito.
- É possível usar os comandos abaixo para continuar ou sair do merge:

```
git merge --abort
```

```
git merge --continue
```

Conflito de merge

Conflitos de merge acontecem quando dois desenvolvedores alteram o mesmo trecho de código ao mesmo tempo.

commit init

dev Bob

```
main() {  
    print("Helo, world!");  
}
```


Conflito de merge

Conflitos de merge acontecem quando dois desenvolvedores alteram o mesmo trecho de código ao mesmo tempo.

commit init

dev Bob

```
main() {  
    print("Helo, world!");  
}
```

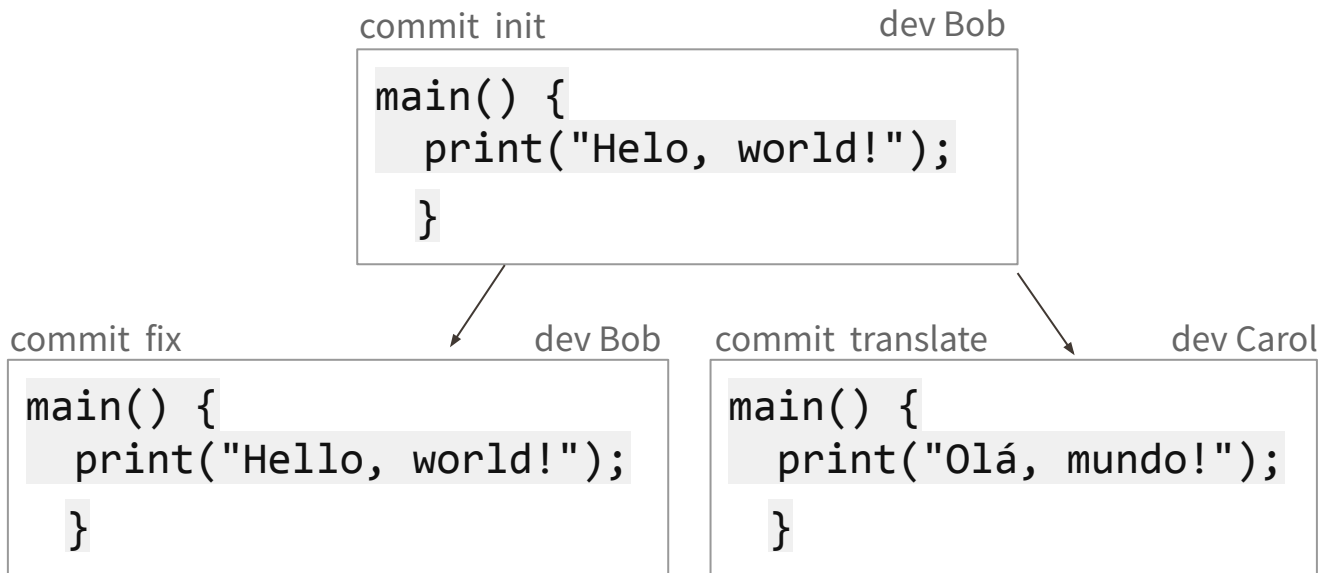
commit translate

dev Carol

```
main() {  
    print("Olá, mundo!");  
}
```

Conflito de merge

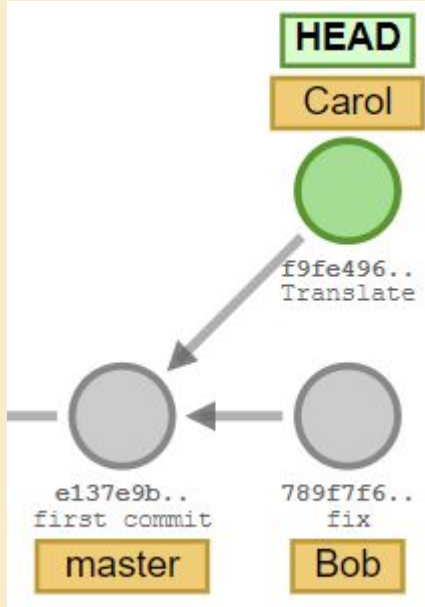
Conflitos de merge acontecem quando dois desenvolvedores alteram o mesmo trecho de código ao mesmo tempo.



Conflito de merge

Se os dois tentarem fazer push com suas alterações não será possível. Vamos supor que bob fez o push primeiro.

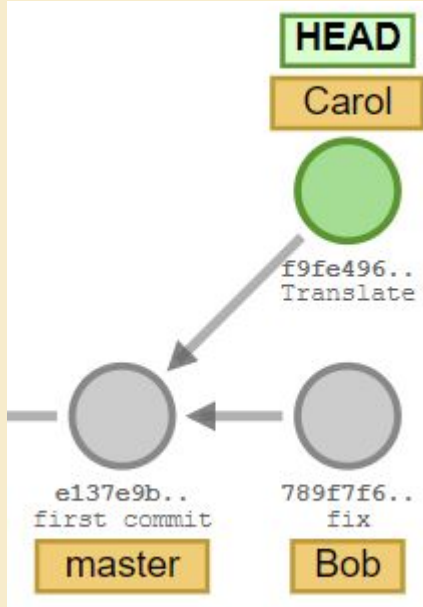
1. push do Bob ✓



Conflito de merge

Se os dois tentarem fazer push com suas alterações não será possível. Vamos supor que bob fez o push primeiro.

1. push do Bob ✓
2. push da Carol 😡

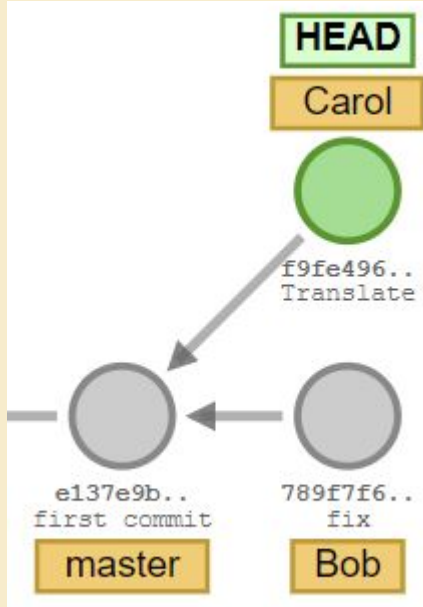


Conflito de merge

Se os dois tentarem fazer push com suas alterações não será possível. Vamos supor que bob fez o push primeiro.

1. push do Bob ✓
2. push da Carol 😡

Carol recebe a informação no terminal que precisa atualizar antes de fazer push.



Conflito de merge

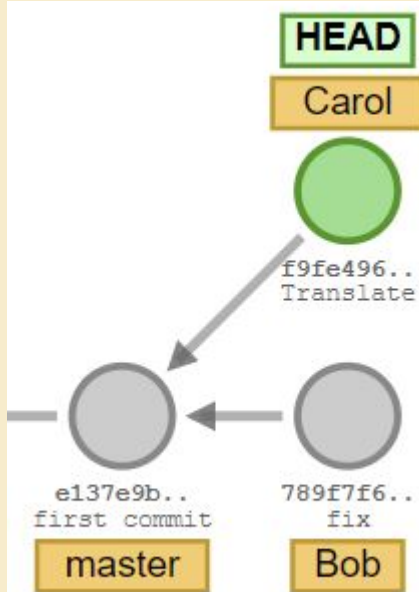
Se os dois tentarem fazer push com suas alterações não será possível. Vamos supor que bob fez o push primeiro.

1. push do Bob ✓
2. push da Carol 😡

Carol recebe a informação no terminal que precisa atualizar antes de fazer push.

3. pull da Carol ⚠

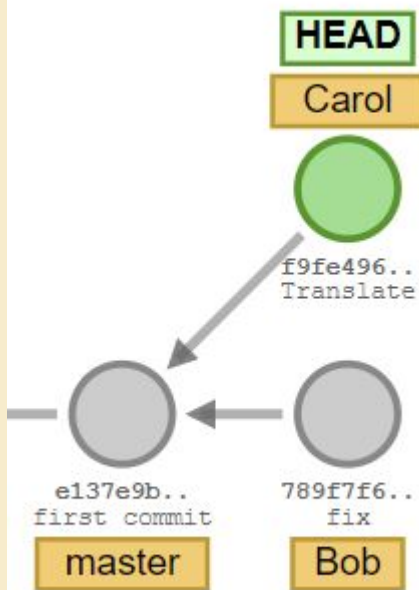
*CONFLICT (content): Merge conflict in arq2
Automatic merge failed; fix conflicts and then
commit the result.*



Conflito de merge

O arquivo em comum agora está assim:

```
main() {  
    <<<<<<< HEAD  
    print("Olá, mundo!");  
    =====  
    print("Hello, world!");  
    >>>>>>> f25bce8fea85a625b891c890a8eca003b723f21b  
}
```

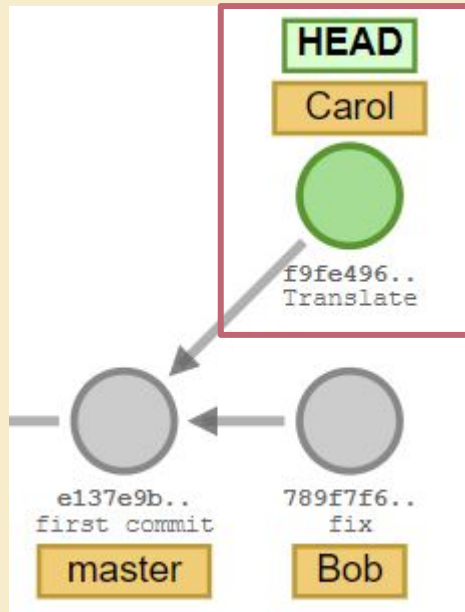


Conflito de merge

O arquivo em comum agora está assim:

```
main() {  
    <<<<<<< HEAD  
    print("Olá, mundo!");  
    =====  
    print("Hello, world!");  
    >>>>>>> f25bce8fea85a625b891c890a8eca003b723f21b  
}
```

HEAD da Carol

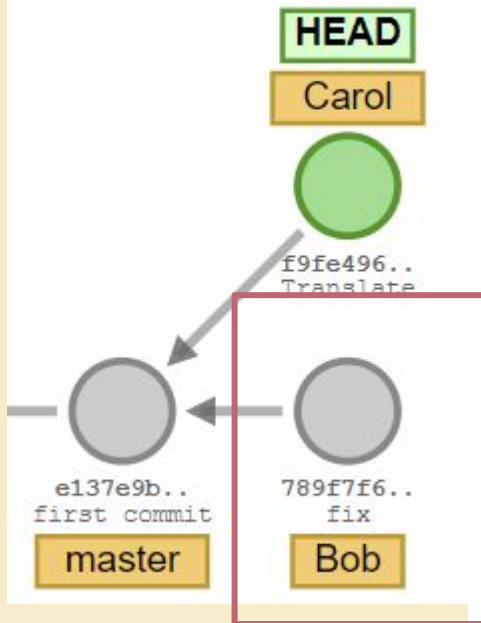


Conflito de merge

O arquivo em comum agora está assim:

```
main() {  
    <<<<<<< HEAD  
    print("Olá, mundo!");  
    =====  
    print("Hello, world!");  
    >>>>>>> f25bce8fea85a625b891c890a8eca003b723f21b  
}
```

HEAD do Repositório remoto



O que fazer? 😞

Conflito de merge

Resolver um conflito é uma **ação manual**. Carol vai tomar a decisão de qual parte deve ficar, removendo as marcações do texto.

```
main() {  
  <<<<<< HEAD  
  print("Olá, mundo!");  
  =====  
  print("Hello, world!");  
  >>>>>> f25bce8fea85a625b891c890a8eca003b723f21b  
}
```

Conflito de merge

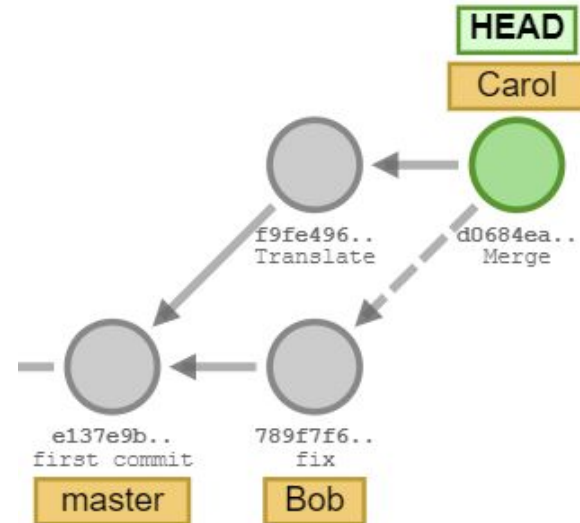
O resultado:

```
main() {  
    print("Olá, mundo!");  
}
```

Conflito de merge

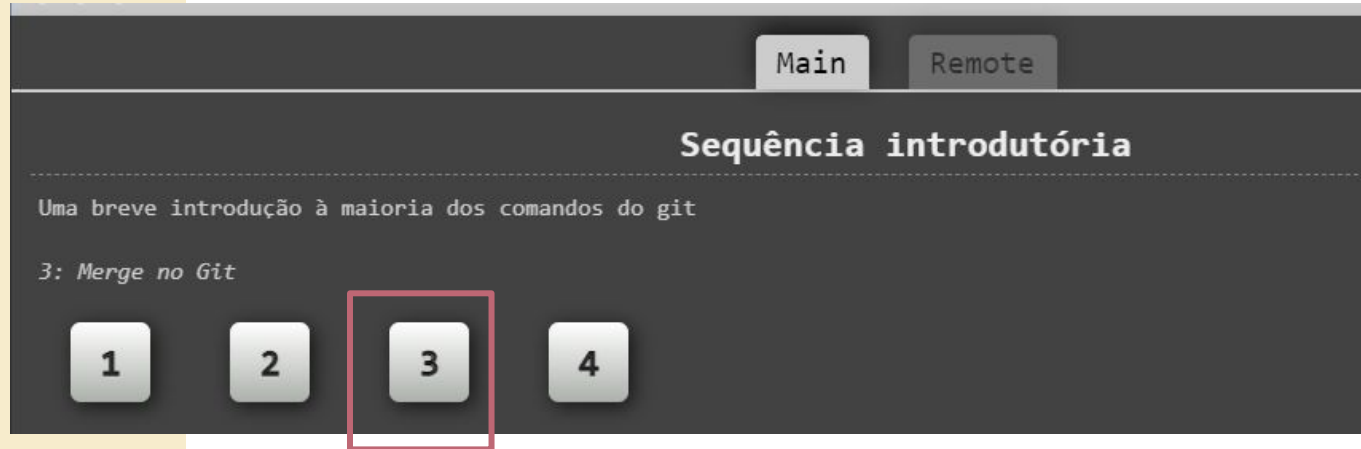
Depois de alterar o arquivo, Carol deve fazer:

1. add ✓
2. commit (se aqui ela não colocar uma mensagem, o Git sugere uma que indique o merge) ✓
3. push ✓



Vamos testar!

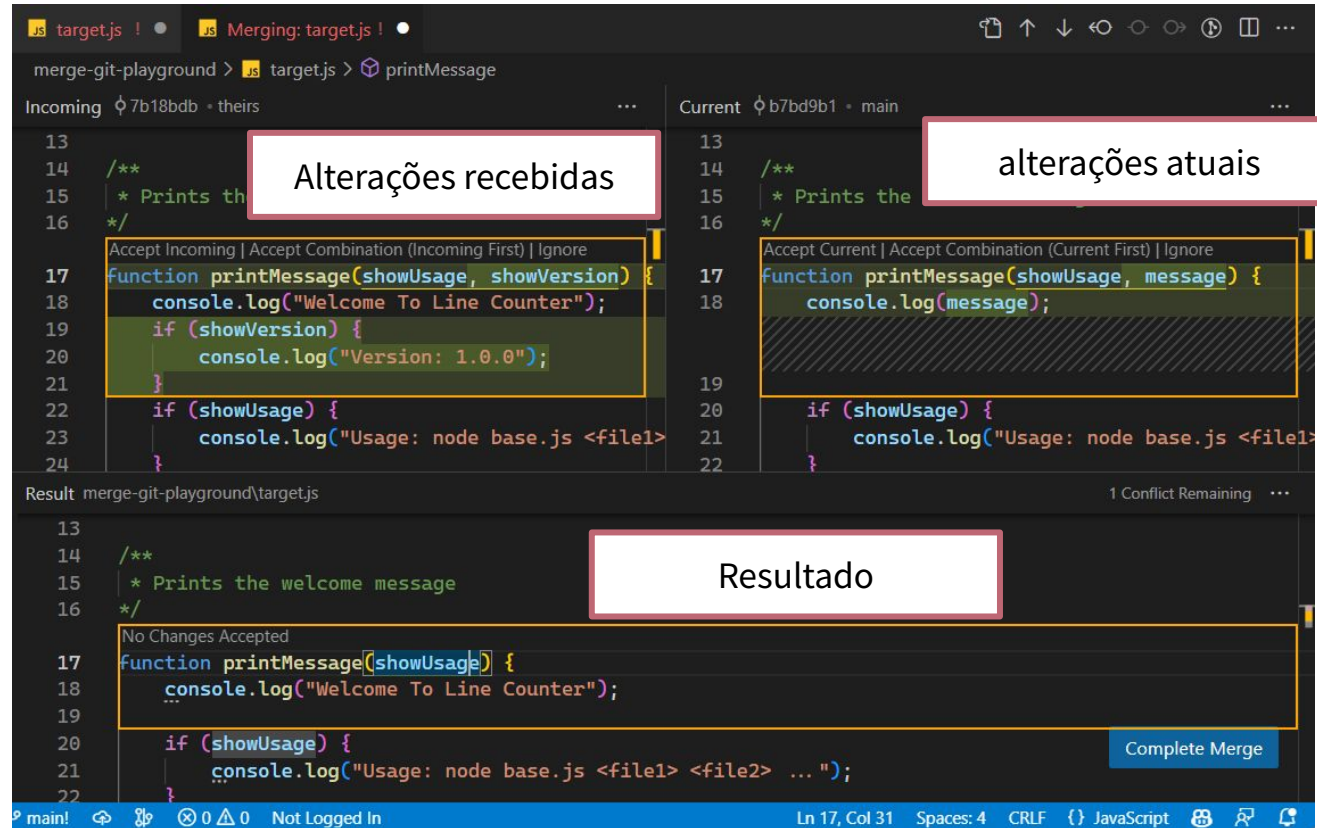
[https://learngitbranching.js.org/?locale=pt BR&NODEM0=](https://learngitbranching.js.org/?locale=pt_BR&NODEM0=)



Conflito no VSCode

Editor de 3 vias

Aproveitem e vejam aqui
outras dicas (que eu
escondi de vocês 😊) para
fazer atividades git no
VSCode.



target.js ! • JS Merging: target.js ! •

merge-git-playground > JS target.js > printMessage

Incoming 7b18bdb • theirs ... Current b7bd9b1 • main ...

13 14 15 16 17 18 19 20 21 22 23 24

13 14 15 16 17 18 19 20 21 22

Result merge-git-playground/target.js 1 Conflict Remaining ...

13 14 15 16 17 18 19 20 21 22

main! 0 0 0 Not Logged In Ln 17, Col 31 Spaces: 4 CRLF {} JavaScript

<https://code.visualstudio.com/docs/sourcecontrol/overview>

git rebase

git rebase

git-rebase - Reaplique os commits em cima do topo de outra base.

git rebase branch nesta-branch

Se **branch** for especificada, o git rebase executará um **git switch** para essa branch antes de fazer qualquer outra coisa. Caso contrário, ele permanecerá no ramo atual.

Se **nesta-branch** não for especificada, utilizará a branch principal padrão.

git rebase

Suponha que o seguinte histórico exista e que o ramo atual seja "topic":

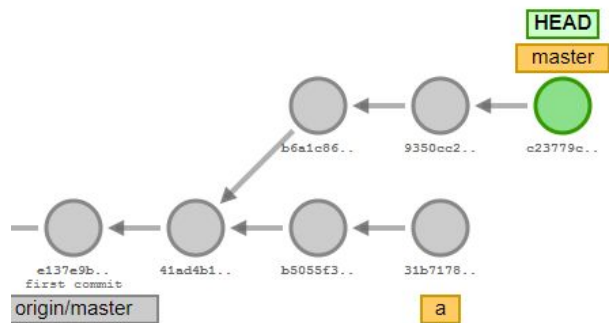
```
A---B---C topic HEAD
/
D---E---F---G master
```

A partir deste ponto, o resultado de um dos seguintes comandos:

- `git rebase master`
- `git rebase master topic`

seria:

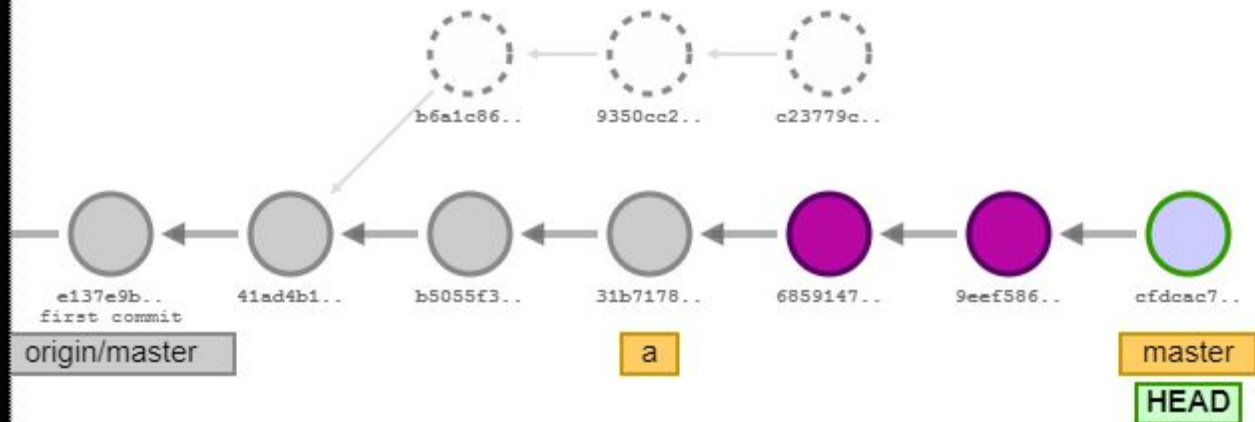
```
A'--B'--C' topic
/
D---E---F---G master HEAD
```



Free Explore with Remote ▾

Have fun!

```
$ git commit
$ git checkout -b a
$ git commit
$ git commit
$ git checkout master
$ git commit
$ git commit
$ git commit
$ git rebase a master
```



git rebase -i

--interactive

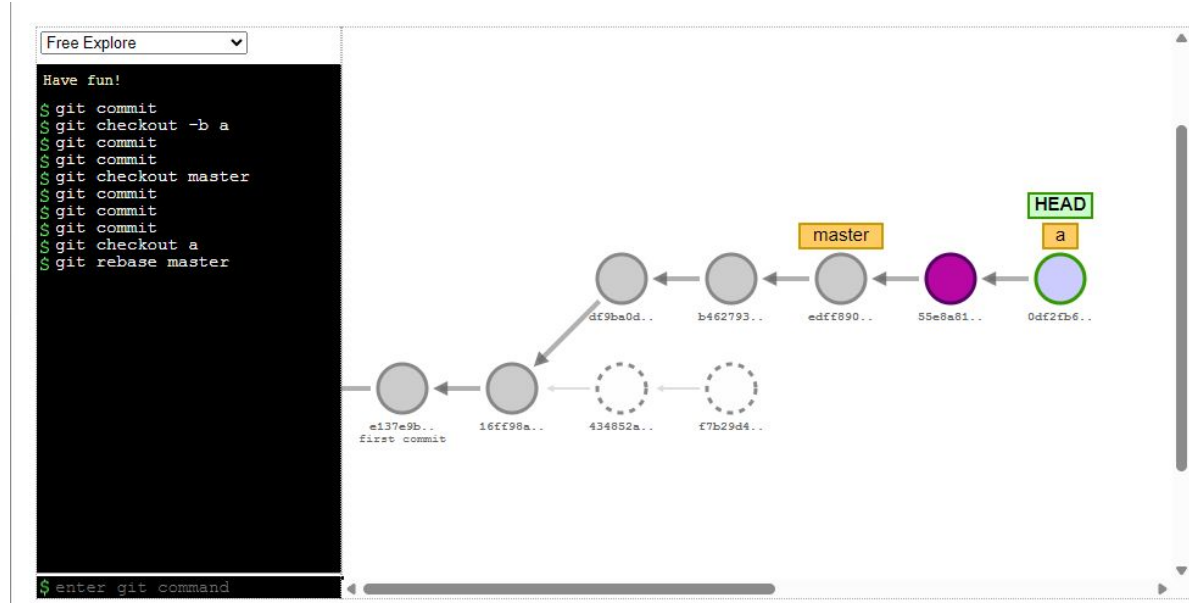
O rebase interativo permite que você altere e reescreva a história dos commits.

Para fazer um git rebase interativo, você pode executar o seguinte comando:

Atenção!

Muitas vezes não é possível reescrever (git push --force) o histórico de commits.

Para manter o histórico de commits e não ter problema em subir para repositórios, ao fazer um rebase esteja na “outra” branch e não da principal (main).



Atenção!

Quando você faz o rebase, você está abandonando commits existentes e criando novos que são semelhantes, mas diferentes.

Se você enviar commits para algum lugar e outras pessoas fizerem o pull e basearem seu trabalho neles, e então você reescrever esses commits com git rebase e enviá-los novamente, seus colaboradores terão que fazer um novo merge em seus trabalhos e as coisas ficarão complicadas quando você tentar puxar o trabalho deles de volta para o seu.

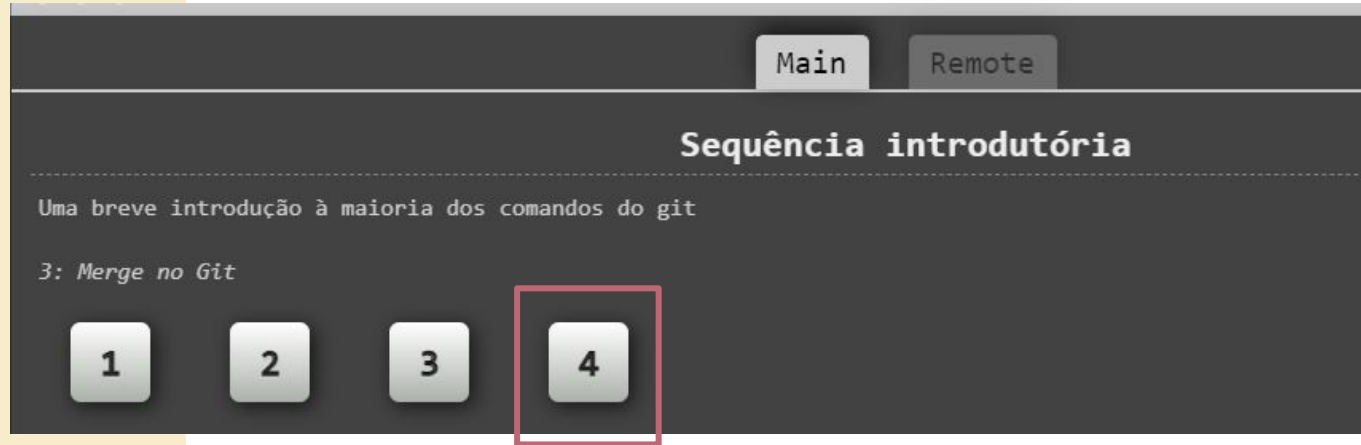
**git pull
--rebase**

ou `git pull -r`

Faz o pull colocando seus commits locais na frente dos commits de onde foi baixado.

Vamos testar!

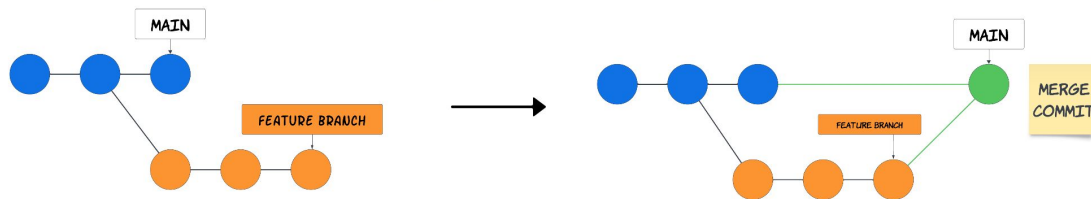
[https://learngitbranching.js.org/?locale=pt BR&NODEM0=](https://learngitbranching.js.org/?locale=pt_BR&NODEM0=)



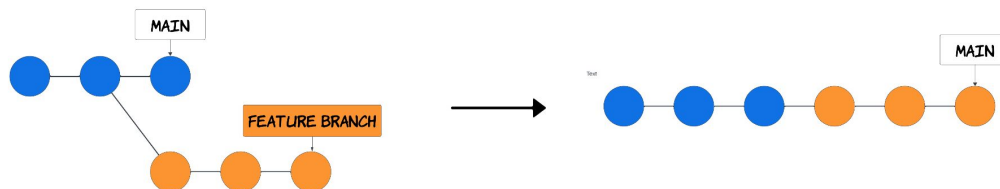
Revisão merge e rebase

Revisão

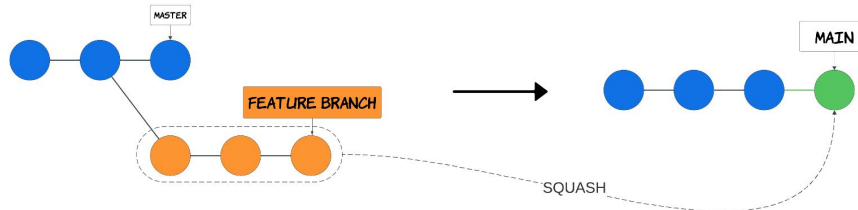
MERGE



REBASE



SQUASH MERGE



Resumo

Merge — Use o recurso ao atualizar um branch de feature compartilhado com outras pessoas para evitar conflitos. Use o recurso merge para atualizar um branch de feature que esteja muito atrasada em relação ao branch principal (main).

Rebase — Use o recurso Rebase ao atualizar um branch de feature que não esteja ainda compartilhado com outros. Isso mantém o histórico limpo e mais fácil de gerenciar.

Squash — Use o recurso Squash ao mesclar alterações do branch de recurso no branch de destino. Isso cria um único commit para todas as alterações, facilitando o rastreamento.

git stash

Guardando pra depois...

git stash - O stashing pega o estado sujo do seu diretório de trabalho — arquivos preparados (staged) e alterações (untracked) — e os salva em uma pilha de alterações inacabadas que podem ser reaplicadas a qualquer momento.

git stash - guarda as alterações em um bloco

git stash pop - pega o último bloco de alterações aplica e remove da lista de stash

git stash list - lista os blocos de stash

git stash apply - aplica um bloco específico

git stash drop - exclui um bloco específico

```
$ git status
```

Changes to be committed:

(use "git reset HEAD <file>..." to unstage)

```
    modified:   index.html
```

Changes not staged for commit:

(use "git add <file>..." to update what will be committed)

(use "git checkout -- <file>..." to discard changes in working directory)

```
    modified:   lib/simplegit.rb
```

```
$ git stash
```

Saved working directory and index state \

"WIP on master: 049d078 added the index file"

HEAD is now at 049d078 added the index file

(To restore them type "git stash apply")

```
$ git status
```

```
# On branch master
```

```
nothing to commit, working directory clean
```

git cherry-pick

git cherry-pick

git cherry-pick <hash-do-commit>

Aplica um commit específico na branch, contanto que seja reconhecido pelo git, isto é, esteja no mesmo repositório.

Não precisa seguir uma linha de commits, é uma aplicação avulsa.

Para adicionar de um outro repositório, você pode adicionar mais um remoto, baixar o conteúdo (fetch), voltar na branch que deseja aplicar

git cherry-pick

Para adicionar de um outro repositório, você pode:

1. adicionar mais um remoto (git remote add),
2. baixar o conteúdo (git fetch),
3. guardar o hash do commit (git log)
4. voltar na branch que deseja aplicar e usar o git cherry-pick

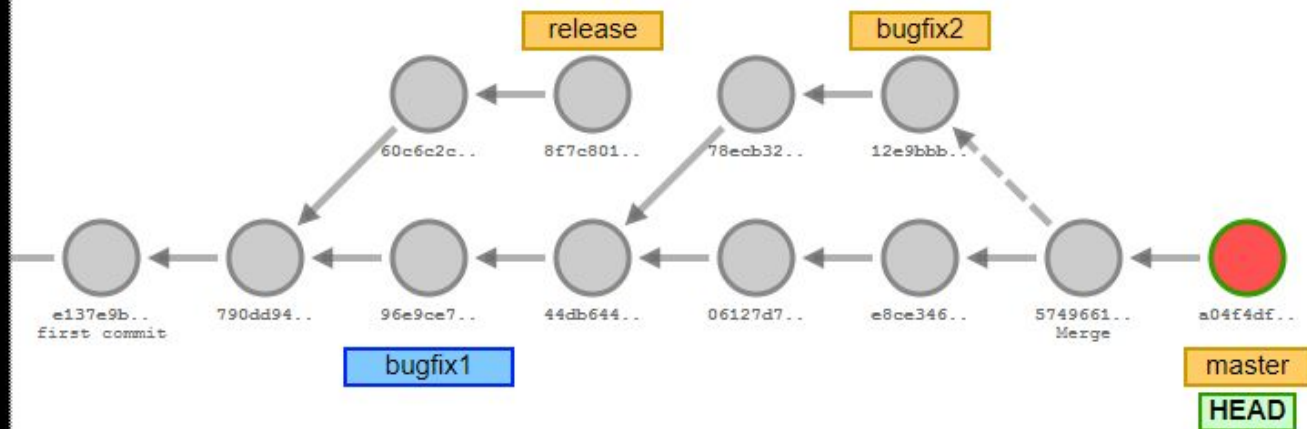
Cherry Pick ▾

Let's pick some commits

```
$ git cherry-pick 60c6c
```

Local Repository

HEAD: master



<https://git-school.github.io/visualizing-git/#cherry-pick>

Vamos testar!

[https://learngitbranching.js.org/?locale=pt BR&NODEMO=](https://learngitbranching.js.org/?locale=pt_BR&NODEMO=)



Dúvidas?
Manda no Discord!



Até a próxima aula!

Lana Mesquita

